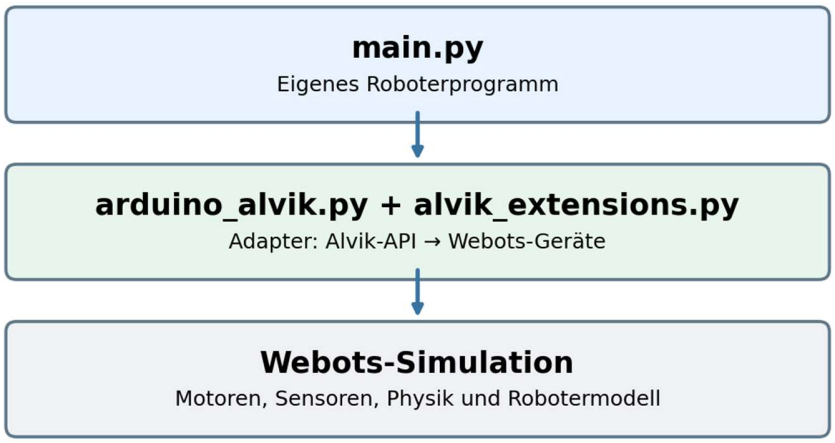
Arduino Alvik in Webots

Projektstruktur verstehen und ein erstes eigenes Python-Programm schreiben

1. Grundidee des Projekts

Das Projekt trennt das eigentliche Roboterprogramm von der Webots-Simulation. Die Alvik-API wird durch einen Python-Adapter auf Webots-Motoren und -Sensoren abgebildet. Dadurch bleibt die Steuerungslogik weitgehend unabhängig vom Simulator.

Ablauf: main.py → arduino_alvik.py / alvik_extensions.py → Webots Controller API → Motoren und Sensoren.




```
alvik.set_wheels_speed(30, 30)
```

Dieser Befehl setzt die Drehzahl des linken und rechten Rades über den Adapter. Direkte Webots-Motorbefehle sind in main.py nicht notwendig.

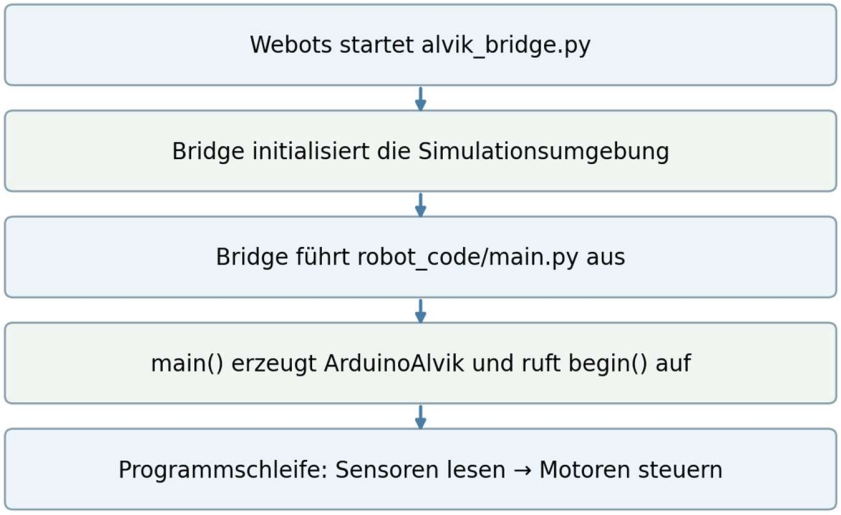
2. Aufgaben der Projektdateien

Datei	Funktion
main.py	Eigenes Roboterprogramm und Verhaltenslogik
alvik_bridge.py	Startet main.py und verbindet Laufzeit mit Webots
arduino_alvik.py	Implementiert die Alvik-Schnittstelle für Webots
alvik_extensions.py	Erweiterungen: Odometrie, Distanzmessung, Fahrbefehle
micropython_time.py	MicroPython-kompatible Zeitfunktionen
beams.py	Visualisiert die fünf Distanzsensorstrahlen
Alvik(3).proto	3D-Robotermodell und Geräte
runtime.ini	Konfiguration des Python-Interpreters
test_*.py / simulation_probe.py	Tests und Messinstrumentierung
RESULTS.md / results.json	Dokumentierte Testergebnisse

Für eigene erste Programme genügt es normalerweise, robot_code/main.py zu bearbeiten. Die Adapterdateien bleiben unverändert.

3. Startablauf

Webots startet den Controller alvik_bridge.py. Die Bridge richtet die Laufzeit ein und führt mit runpy.run_path(...) die Datei robot_code/main.py aus. Dort wird ArduinoAlvik() erzeugt und mit begin() initialisiert. Danach läuft die Programmlogik.




```
runpy.run_path(
    str(Path(__file__).resolve().parents[2] / 'robot_code/main.py'),
    run_name='__main__'
)
```

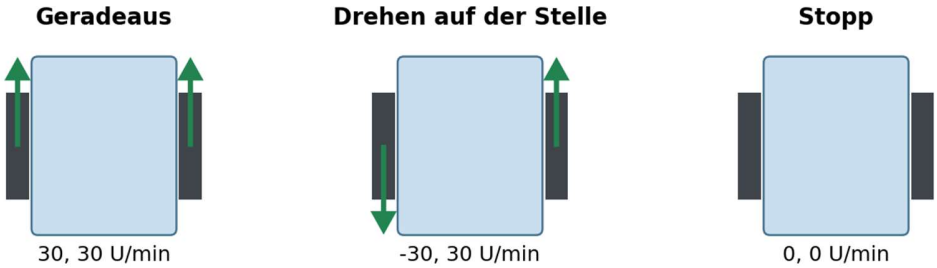
Wichtig: sleep_ms() schaltet in dieser Umgebung auch die Webots-Physik weiter. Ohne Simulationsschritte würden sich Sensorwerte und Roboterposition nicht aktualisieren.

4. Aufbau der vorhandenen main.py

Bestandteil	Aufgabe
Konstanten	Grenzabstände, Winkel und Parameter
calculate_center()	Berechnet die Abweichung von der Linie
run_line_follower()	P-Regler zur Linienverfolgung
Klasse Detour	Zustandsautomat für die Hindernisumfahrung
main()	Initialisierung und Programmschleife

Der Zustandsautomat nutzt unter anderem FOLLOW, TURN_OUT, OUT, TURN_PARALLEL, PASS, TURN_IN, REJOIN, ALIGN und STOP. Für einen ersten Versuch ist diese Logik nicht erforderlich.

5. Erstes Programm: Drei Sekunden geradeaus fahren



Das Programm initialisiert den Alvik, wartet zwei Sekunden, fährt drei Sekunden mit 30 U/min und hält anschließend an. Sichern Sie vorher die bisherige main.py.

```
from arduino_alvik import ArduinoAlvik
from time import sleep_ms

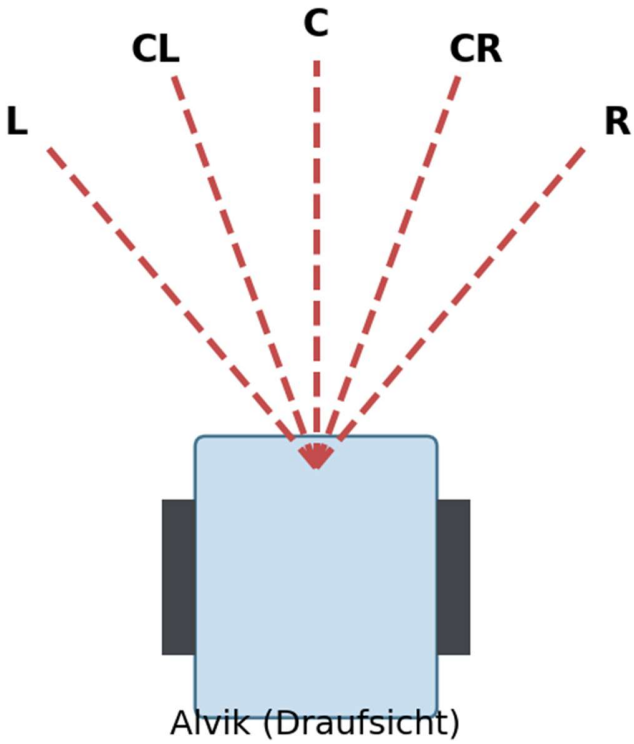

def main():
    alvik = ArduinoAlvik()
    alvik.begin()
    print("Alvik ist bereit")

    try:
        sleep_ms(2000)
        alvik.set_wheels_speed(30, 30)
        sleep_ms(3000)
    finally:
        alvik.brake()
        print("Alvik gestoppt")


if __name__ == "__main__":
    main()
```

ArduinoAlvik() erzeugt das Roboterobjekt; begin() initialisiert Motoren, Sensoren und Tastatur. set_wheels_speed(links, rechts) verwendet standardmäßig U/min. Mit (30, 30) fährt der Alvik geradeaus, mit (-30, 30) dreht er auf der Stelle, mit (0, 0) stoppt er.

6. Zweites Programm: Hindernis erkennen



get_distance() liefert fünf Abstandswerte in Zentimetern: links, mitte-links, mitte, mitte-rechts, rechts. Für die Frontüberwachung wird der kleinste der drei mittleren Werte verwendet.

```
from arduino_alvik import ArduinoAlvik
from time import sleep_ms


def main():
    alvik = ArduinoAlvik()
    alvik.begin()
    print("Hinderniserkennung gestartet")

    try:
        while True:
            abstaende = alvik.get_distance()
            vorne = min(abstaende[1:4])
            print(f"Abstand vorne: {vorne:.1f} cm")

            if vorne < 10:
                alvik.brake()
                print("Hindernis erkannt!")
            else:
                alvik.set_wheels_speed(30, 30)

            sleep_ms(100)
    finally:
        alvik.brake()
```

```
if __name__ == "__main__":
    main()
```

Die Abfrage erfolgt alle 100 ms. Bei weniger als 10 cm stoppt der Roboter; sobald der Weg wieder frei ist, fährt er weiter. Das ist eine reaktive Steuerung, noch kein Zustandsautomat.

7. Eigenes Programm in Webots starten

- Vorhandene .wbt-Welt in Webots R2025a öffnen.
- Prüfen, ob dem Roboter der Controller alvik_bridge zugeordnet ist.
- robot_code/main.py sichern, durch das Beispiel ersetzen und speichern.
- File → Reload World wählen, damit der Controller neu startet.
- Simulation mit Play starten und die Konsole beobachten.

Hinweis: In der hochgeladenen PROTO-Datei ist als Standardcontroller alvik_model_check eingetragen. Die tatsächliche Controller-Zuordnung in der Welt ist deshalb zu prüfen. runtime.ini legt keinen projektspezifischen Python-Befehl fest.

8. Wichtige Funktionen des Simulationsadapters

Befehl	Bedeutung
alvik.begin()	Roboter initialisieren
alvik.set_wheels_speed(30, 30)	Raddrehzahlen in U/min
alvik.drive(5, 0)	5 cm/s vorwärts
alvik.drive(0, 45)	45 Grad/s drehen
alvik.brake()	Anhalten
alvik.get_distance()	Fünf Frontabstände
alvik.get_line_sensors()	Drei Liniensensorwerte
alvik.get_pose()	Odometrie (x, y, theta)
alvik.reset_pose()	Odometrie zurücksetzen
alvik.get_wheels_speed()	Gemessene Raddrehzahlen
alvik.get_touch_ok()	R oder Enter
alvik.get_touch_cancel()	X oder Leertaste
sleep_ms(100)	100 ms Simulationszeit

Nicht jede Erweiterung des Webots-Adapters ist automatisch Bestandteil der offiziellen Arduino-Alvik-MicroPython-API. Die Portierung auf echte Hardware erfordert gegebenenfalls Anpassungen.

9. Vorschlag für die ersten Praktikumsaufgaben

- Übung 1 – Motorsteuerung: Geradeaus fahren, etwa 90° drehen, anhalten.
- Übung 2 – Sensorik: Bei einem Hindernis unter 10 cm stoppen.
- Übung 3 – Odometrie: 80 cm fahren und die Strecke mit get_pose() überwachen.

Fazit: Die eigenen Programme gehören in main.py. Bridge, Adapter und Webots-Modell sollten zunächst unverändert bleiben.